

Installation Guide (from scratch)

This guide takes you from nothing installed to a working inspection mission, in plain steps. You do not need to be a ROS expert — just follow it top to bottom. Where a step is easy to get wrong, there's a ✓ Success check so you know it worked before moving on.

What you're building

A robot (Boston Dynamics Spot) walks around a site, builds a map with a depth camera, drives itself to the best spots, and takes 3D laser scans with a Trimble X7 scanner. You supervise it from a laptop.

It runs on two computers, and there's a good reason for the split:

	Jetson (rides on Spot)	Windows laptop (you hold it)
Runs	The robot brain: mapping, planning, driving Spot	Trimble Perspective + a small "bridge" app
Talks to	Spot and the depth camera	The Trimble X7 scanner, and the Jetson
Why it exists	It's the autonomy computer, on the robot	Perspective only runs on Windows — that's the whole reason there are two computers

The laptop does no mapping or driving. It's just the scanner's remote and your console. If it drops offline mid-mission, the robot keeps mapping and walking — it just can't take scans until it's back.

Before you begin

Hardware you need:

- Spot robot, powered, with its tablet controller.
- An NVIDIA Jetson Orin (Nano/NX/AGX) mounted on Spot as a payload.
- A depth camera (Luxonis Oak-D Pro), USB-connected to the Jetson.
- A Trimble X7 scanner (with its tripod).
- A Windows laptop with Trimble Perspective installed.
- A Beryl AX travel router (creates the field Wi-Fi network).
- A USB Wi-Fi adapter for the laptop (so it can be on the scanner's Wi-Fi and the robot network at once). See the network guide below.

Accounts / software you need:

- The GitHub repository (this project). If it's private, make sure your GitHub account has access before you start.
- Spot login: its IP address, username, and password.
- Trimble Perspective already installed and paired with your X7 — confirm you can take a scan from Perspective alone before adding any of our software.

Roughly how it goes (about half a day the first time):

1. Jetson — download the code and install the robot software (most of the work).
2. Windows laptop — download the code and install the bridge app.
3. Network — put both computers on the Beryl Wi-Fi so they can talk.
4. First run — test with no hardware, then a supervised real mission.

Tip: Do the whole no-hardware test at the end of Part 1 first. If that runs, your software is good, and you've only got wiring and network left.

What gets installed where (read this first)

This is the most common source of confusion, so keep it straight:

The entire robot pipeline — ROS, mapping, planning, Spot, the camera — runs on the JETSON. The Windows laptop runs ONLY the scanner console (Trimble Perspective + the bridge app). No part of the ROS pipeline runs on Windows.

Software	Jetson (Ubuntu, on Spot)	Windows laptop
ROS 2 Jazzy (+ ros-jazzy-* packages)	✓	✗
Python mapping libs (numpy, opencv-python, pyyaml)	✓	✗
Spot SDK (bosdyn-client) + scan libs (laspy, lazrs)	✓	✗
Depth-camera driver (depthai-ros)	✓	✗
colcon build of this project	✓	✗
Trimble Perspective (vendor app)	✗	✓
Python 3.12 for Windows + bridge-app deps	✗	✓
This project's code (git clone)	✓ full workspace	✓ only the tools/trimble_perspective_bridge folder is used

In one line: anything about ROS / Spot / the camera → Jetson only. Anything about Trimble Perspective → Windows only. The two machines just message each other over the network (Part 3).

Part 1 — Set up the Jetson (the robot's computer)

Do all of this on the Jetson (SSH in, or plug in a monitor + keyboard).

1.0 Check what you have, and install the basics

Every later step depends on your Jetson's software version, so check first, and install the tools you'll need to download the code:

```
cat /etc/nv_tegra_release          # shows your JetPack/L4T version
sudo apt update
sudo apt install -y python3-pip git curl
```

This guide assumes JetPack 6 (based on Ubuntu 24.04). If yours is older (Ubuntu 22.04), see the note in step 1.2.

1.1 Download this project (the code)

What this is: the robot software lives in a GitHub repository. You "clone" (download) it into a folder called `ros2_ws` in your home directory — that folder is your workspace, and the rest of this guide runs commands from inside it.

```
cd ~
git clone https://github.com/woopers6/robot-inspection.git ros2_ws
cd ros2_ws
```

✓ Success check:

```
ls          # you should see: src  scripts  config  docs  tools  README.md ...
```

If git clone asks for a username/password, the repo is private — log in with your GitHub account (or a personal access token). If you were given a .zip instead, unzip it to ~/ros2_ws and continue.

1.2 Install ROS 2 Jazzy (the robot framework)

What this is: ROS 2 is the plumbing all the robot programs use to talk to each other. "Jazzy" is the specific version. Copy-paste this whole block:

```
sudo apt install -y software-properties-common
sudo add-apt-repository universe
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

sudo apt update
sudo apt install -y ros-jazzy-ros-base ros-dev-tools
sudo apt install -y \
  ros-jazzy-cv-bridge \
  ros-jazzy-vision-msgs \
  ros-jazzy-nav2-msgs \
  ros-jazzy-tf2-ros \
  ros-jazzy-tf2-geometry-msgs \
  ros-jazzy-message-filters \
  ros-jazzy-rviz2

# Make ROS available in every new terminal automatically:
echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc
source /opt/ros/jazzy/setup.bash
```

✓ Success check:

```
ros2 --help      # should print ROS command help, not "command not found"
```

On Ubuntu 22.04? Jazzy has no packages for it. Either update to a JetPack release built on 24.04, or run everything inside the official ros:jazzy Docker container (docker run --runtime nvidia ...). Don't try to compile ROS from source on the Jetson — it takes hours and breaks on updates.

1.3 Install the Python libraries

What this is: the mapping and planning are plain math (NumPy) — there's no AI model to install, no GPU setup. Just a few Python packages:

```
pip3 install opencv-python numpy pyyaml
```

1.4 Install the Spot SDK and scan libraries

What this is: the code that lets the Jetson talk to Spot and read .las scan files. These are listed in requirements-field.txt (Spot's bosdyn-client, plus laspy/lazrs):

```
cd ~/ros2_ws
pip3 install --user -r requirements-field.txt
```

1.5 Install the depth-camera driver

What this is: the program that turns the Oak-D camera into ROS data. Our software subscribes to the camera; it doesn't start it for you.

```
sudo apt install -y ros-jazzy-depthai-ros
# Let the Jetson talk to the camera over USB without root:
echo 'SUBSYSTEM=="usb", ATTRS{idVendor}=="03e7", MODE="0666" \
| sudo tee /etc/udev/rules.d/80-movidius.rules
sudo udevadm control --reload-rules && sudo udevadm trigger
```

The map needs a point cloud (3D dots), not just a flat depth image. If your camera driver only gives a depth image, also install this and run its `point_cloud_xyz` node to create the point-cloud topic:

```
sudo apt install -y ros-jazzy-depth-image-proc
```

Localization is the camera, not Spot. The mission gets the robot's position entirely from the depth camera's visual-inertial odometry (topic `/oak/odom`) — Spot's leg/joint-encoder odometry is not used at all. So your camera launch must also publish odometry: enable the OAK's on-device VIO, or run a VIO / RTAB-Map node that outputs `/oak/odom`. Check it's alive with `ros2 topic hz /oak/odom` before a mission. (If it isn't publishing, the robot won't be localized and the map won't build.)

1.6 (Optional) Fused localization

Skip this — leave it off. Localization comes from the depth camera alone (step 1.5). This optional EKF would fuse Spot's kinematic pose + an IMU back into the estimate, which reintroduces Spot's joint-encoder odometry — the opposite of the camera-only setup. Only enable it if you deliberately decide you want Spot's pose in the mix again:

```
sudo apt install -y ros-jazzy-robot-localization # only if FUSED_LOCALIZATION=true
```

Details (the `ekf.yaml` inputs, the `navX` IMU bridge) are in `launch/fused_localization.launch.xml`. Keep `FUSED_LOCALIZATION=false`.

1.7 Build the software

What this is: "building" compiles our code into something ROS can run. Run this from the workspace folder:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash # load what you just built into this terminal
```

✓ Success check: the build ends with Summary: N packages finished and no red Failed lines.

Every new terminal needs `source install/setup.bash` before running the robot. To automate it: `echo 'source ~/ros2_ws/install/setup.bash' >> ~/.bashrc`

1.8 Configure your site settings

What this is: one file holds all your site-specific values (Spot's password, the laptop's address, etc.). Copy the template and edit it:

```
cd ~/ros2_ws
cp config/field.env.example config/field.env
nano config/field.env # or any editor
```

At minimum, set:

- `SPOT_IP`, `SPOT_USERNAME`, `SPOT_PASSWORD` — how to reach and log into Spot.
- `TRIMBLE_WINDOWS_URL` — the laptop's address, e.g. `http://10.0.0.20:8765` (you'll finalize this in Part 3).

Then confirm the camera topic names match your driver (don't trust the defaults):

```
ros2 topic list | grep -E "rgb|depth|points"
```

If the names differ, update the `IMAGE_TOPIC` / `MAP_DEPTH_POINTS_TOPIC` lines in `config/field.env` to match.

1.9 Prove it works with NO hardware

Before touching the robot, run the whole loop in a pure-software simulation. This is the single most useful check in the guide:

```
./scripts/run_field.sh mission --sim synthetic
```

✓ Success check: a window (RViz) opens and you watch a simulated robot explore a map, drive to scan spots, and return "home." If this runs, your Jetson software is fully installed and correct. Press Ctrl-C to stop.

You can also run the built-in preflight, which checks the Spot SDK and whether Spot answers on the network:

```
./scripts/field_preflight.sh mission
```

Part 2 — Set up the Windows laptop (the scanner console)

This machine runs Trimble Perspective (which you already have) plus our small bridge app — and nothing from Part 1: no ROS, no Spot SDK, no camera driver, no colcon build. Just Perspective, Python 3.12, and the bridge.

2.0 Get the code onto the laptop

The bridge app is part of the same project, so you need the code here too. Two easy ways:

- Easiest: on the GitHub page, click the green Code ► Download ZIP, then unzip it (e.g. to C:\robot-inspection).
- Or with Git: install Git for Windows, then in a folder run `git clone https://github.com/woopers6/robot-inspection.git`.

2.1 Prerequisites

1. Trimble Perspective installed and already taking scans from your X7. Get this working on its own first — debugging the scanner and our software at the same time is painful.
2. Python 3.12 for Windows from python.org. During install, tick "Add python.exe to PATH."

2.2 Install and start the bridge app

In a Command Prompt, go into the folder you just downloaded, then:

```
cd tools\trimble_perspective_bridge
"Install Windows Dependencies.bat"
py windows_app.py
```

(or just double-click Launch Trimble Bridge.bat.)

✓ Success check: a window titled "TxDOT Digital Twin Inspection Console" opens, and a browser dashboard appears at <http://127.0.0.1:8765>.

2.3 Point it at the Jetson

In the app's Settings tab:

Setting	Value
jetson_host	the Jetson's IP, e.g. 10.0.0.10
jetson_user	your Jetson login name
jetson_workspace	~/ros2_ws
jetson_mode	mission
windows_advertise_host	the laptop's robot-LAN IP, e.g. 10.0.0.20
export_dir	Perspective's scan export folder
mission_output_dir	where the final E57 file is saved
auto_transfer	off — scans stay on the scanner's SD card

2.4 Open the Windows firewall

Windows silently blocks the Jetson otherwise. In an Administrator PowerShell:

```
New-NetFirewallRule -DisplayName "Trimble Bridge" -Direction Inbound `
  -LocalPort 8765 -Protocol TCP -Action Allow
```

Part 3 — Connect the network

The Jetson and the laptop have to be on the same Wi-Fi network to talk. The Beryl AX router creates that network.

Full step-by-step (with a diagram) is in docs/field_network_setup.docx — follow that for the router, the Jetson Wi-Fi, and the laptop's USB Wi-Fi adapter. The short version:

- Three separate Wi-Fi/wired networks that must not overlap:
- Spot ↔ Jetson: wired over Spot's payload port (192.168.50.x)
- Jetson ↔ laptop: the Beryl Wi-Fi (10.0.0.0/24)
- laptop ↔ X7: the scanner's own Wi-Fi
- Give fixed addresses: Jetson 10.0.0.10, laptop 10.0.0.20.
- The laptop needs two Wi-Fi radios (its built-in one on the X7, the USB adapter on the Beryl) because one radio can't be on two Wi-Fis at once.

What talks to what:

From → To	Port	Purpose
Jetson → Spot	443	robot pose + motion (Spot SDK)
Jetson → laptop	8765	asks for scans, checks status
laptop → Jetson	22 (SSH)	starts/stops the robot software
laptop ↔ X7	scanner Wi-Fi	Perspective runs the scan

Set the same ROS_DOMAIN_ID (default 42) on the Jetson.

✓ Success check — test both directions before the robot leaves the bench:

```
# On the Jetson
ping 10.0.0.20 && curl http://10.0.0.20:8765/status

# On the laptop
ssh <user>@10.0.0.10 "echo ok"
```

Both should succeed (the curl returns some JSON; the ssh prints ok).

Part 4 — Your first mission

Always do a supervised, no-motion run first.

1. Laptop: open Perspective, then start the bridge app.
2. Jetson: run the mission with motion turned off:

```
ROBOT_GOAL_BRIDGE=false ./scripts/run_field.sh mission
```

The software plans and suggests where to go but does not drive Spot. Walk Spot yourself with the tablet and watch the map fill in.

1. Confirm the pieces are alive:

```
ros2 topic hz /depth/points          # camera is feeding the map
ros2 topic echo /mission/state --once # mission is progressing
```

1. How scanning works: when the robot reaches a chosen spot it parks and waits — it will not move on by itself. You take the scan in Perspective, then press "Next scan location" in the bridge app to release it to the next spot. (This is by design — the operator paces the scans.)
2. Only when all of that is clean, enable real driving:

```
ROBOT_GOAL_BRIDGE=true ROBOT_GOAL_BACKEND=spot_sdk ./scripts/run_field.sh mission
```

Keep the tablet E-Stop in your hand.

1. End of mission: robot returns to its start → state shows AWAITING_UPLOAD → copy the E57 file off the X7's SD card → press "Upload E57 + Finish" in the app.

Everyday use (after it's all installed)

```
# no-hardware demo (great for showing the flow):
./scripts/run_field.sh mission --sim synthetic

# real mission, no driving (supervised):
ROBOT_GOAL_BRIDGE=false ./scripts/run_field.sh mission

# real mission, autonomous driving (E-Stop in hand):
ROBOT_GOAL_BRIDGE=true ROBOT_GOAL_BACKEND=spot_sdk ./scripts/run_field.sh mission
```

Other modes: `./scripts/run_field.sh demo` (real robot, invented site, open area only) and `./scripts/run_field.sh slam` (camera-only mapping, no Spot or Trimble). Run `./scripts/run_field.sh --help` for all options.

Troubleshooting

git clone asks for a password / "repository not found" — the repo is private and your GitHub account needs access, or you mistyped the URL. Log in with a personal access token, or use the Download-ZIP option.

ros2: command not found — you didn't load ROS in this terminal. Run `source /opt/ros/jazzy/setup.bash` (and `source ~/ros2_ws/install/setup.bash`).

colcon build fails — usually a missing `ros-jazzy-*` package from step 1.2, or you're on Ubuntu 22.04 (see the note in 1.2). Read the first red error; it names the missing piece.

The robot never moves to the next scan spot — that's expected. It waits for you to press "Next scan location" in the bridge app. If pressing it does nothing, the Jetson can't reach the laptop: on the Jetson, check `curl http://10.0.0.20:8765/status` returns JSON with a `scans_completed` number. If that fails, revisit Part 3 (network) and the firewall (2.4).

Jetson can't reach the laptop / no wlan0 — a network problem. Work through `docs/field_network_setup.docx`; confirm both are on the Beryl Wi-Fi with the right IPs.

A hole in the map right under the robot — the depth camera isn't feeding the map. Check the point-cloud topic exists (ros2 topic `hz /depth/points`) and that your `MAP_DEPTH_POINTS_TOPIC` in `field.env` matches ros2 topic list.

Robot returns but misses its start spot — the camera's visual-inertial odometry has drifted. Check `/oak/odom` is healthy (ros2 topic `hz /oak/odom`) and that the camera has enough texture/light for VIO; poor VIO is the usual cause. (Localization is camera-only by design — Spot's odometry is not used.)

Spot doesn't answer — wrong `SPOT_IP`, or it's not on the network yet. Run `./scripts/field_preflight.sh mission` and ping `<SPOT_IP>`.

Mini-glossary

- Repository (repo) — the project's code, stored on GitHub. You "clone" it to download a copy.
- Workspace — the `~/ros2_ws` folder that holds the code and the built output.
- ROS 2 / Jazzy — the framework the robot programs run on. "Jazzy" is the version.
- colcon build — compiles this project's code so ROS can run it.
- Occupancy map — the 2D map of free vs. blocked space the robot builds from the depth camera.
- Topic — a named channel programs use to share data (e.g. `/depth/points`).
- TF — ROS's bookkeeping of where each part is in space (camera vs. robot vs. map).

- Payload port / GXP — the port on Spot's back that powers and networks the Jetson.
- field.env — your one settings file (config/field.env).
- Bridge app — the Windows program that relays scan requests between the Jetson and Trimble Perspective.